

LABORATÓRIO DE ENGENHARIA DE SOFTWARE

Nexus Estates

Guia de Execução, Arquitetura e Testes

Gestão centralizada de reservas e ocupação para imóveis. Um ecossistema de microserviços focado em consistência de dados, prevenção de erros de reserva e integração simulada com plataformas externas.

Repositório GitHub:

<https://github.com/kanekitakitos/Nexus-Estates>

Docentes

José Barateiro
Marco Giunti

Estudantes

Brandon Mejia – 79261
Luís Moreira – 81432
Miguel Correia – 71369
Tiago Antunes – 76920

Conteúdo

1	Stack tecnológica	2
1.1	Backend	2
1.2	Frontend	2
1.3	Infraestrutura	2
1.4	Justificação das Escolhas	2
2	Equipa, Gestão e Integração Contínua	3
2.1	Papéis da Equipa	3
2.2	Gestão de Projeto (SCRUM e Requisitos)	3
2.3	Integração Contínua (CI) e Quality Gates	3
3	Objetivo	4
4	Como correr o projeto (Docker – Deploy em Produção)	5
4.1	Comando de Deploy	5
4.2	Acessos e Credenciais	5
4.3	Gestão do Ciclo de Vida	5
4.4	Arquitetura de Rede e Segurança	6
5	Ambiente de Desenvolvimento (DEV)	6
5.1	Infraestrutura Base	6
5.2	Backend e Frontend	6
6	Padrões de Projeto e Práticas (Backend)	6
6.1	Separação por Camadas (MVC + Service + Repository)	6
6.2	Proxy Pattern (HTTP Declarativo entre Microserviços)	7
6.3	Strategy Pattern (Ex.: Faturação no finance-service)	7
6.4	Resiliência (Circuit Breaker + Retry)	7
6.5	Eventos Assíncronos e Consistência Eventual	7
7	Testes (backend)	9
7.1	api-gateway	9
7.2	booking-service	9
7.3	property-service	9
7.4	sync-service	9
7.5	finance-service	9
7.6	user-service	9
7.7	Como correr os testes (backend)	10
8	Testes (frontend) — integração/E2E (Playwright)	10
8.1	Como correr testes do frontend	10
	Referências	11

1 Stack tecnológica

1.1 Backend

- Java 23, Spring Boot 3.3.x
- Spring Cloud Gateway (API Gateway)
- Spring Data JPA + PostgreSQL 16 (persistência)
- Flyway (migrações)
- RabbitMQ (AMQP) para eventos assíncronos
- SpringDoc OpenAPI/Swagger UI
- Resilience4j (Circuit Breaker/Retry) no **sync-service** para integrações externas
- Integrações: Stripe (pagamentos/webhooks), Cloudinary (media), Ably (chat/webhooks)

1.2 Frontend

- Next.js (App Router) + TypeScript
- Bun (runtime + package manager)
- Tailwind CSS
- Axios (camada de comunicação com a API via Gateway)
- Playwright (testes E2E/integração)

1.3 Infraestrutura

- Docker + Docker Compose (orquestração local/deploy)
- Postgres (container único com múltiplas bases de dados)
- RabbitMQ (broker + management)

1.4 Justificação das Escolhas

- **Java (Spring Boot) vs. Node.js:** Optámos por Java no backend devido ao nosso maior nível de conhecimento e controlo sobre a linguagem para gerir a lógica complexa dos microserviços.
- **Next.js vs. Express/React nativo:** Embora tenhamos experiência com Express, escolhemos o Next.js pela enorme facilidade de uso, convenções integradas de roteamento e otimização imediata.
- **TypeScript vs. JavaScript:** Escolhemos TypeScript em todo o frontend porque a tipagem estática previne erros em *runtime* e melhora a experiência de desenvolvimento e manutenção.
- **Tailwind CSS vs. CSS Tradicional:** O Tailwind foi selecionado porque a sua abordagem *utility-first* acelera drasticamente a criação da UI sem termos de sair dos ficheiros dos componentes.

2 Equipa, Gestão e Integração Contínua

2.1 Papéis da Equipa

A equipa dividiu as responsabilidades e o foco de desenvolvimento da seguinte forma:

- **Brandon Mejia:** *Fullstack*
- **Tiago Antunes:** *Frontend*
- **Luís Moreira (Daniel):** *Backend*
- **Miguel Correia:** *Backend*

2.2 Gestão de Projeto (SCRUM e Requisitos)

O projeto segue a metodologia ágil SCRUM. Optámos por não duplicar documentação neste relatório, uma vez que o levantamento de requisitos, a definição de atores, a gestão do *backlog* (com *user stories*) e o planeamento dos *sprints* estão totalmente centralizados e documentados no **Jira**.

Ao nível de código, utilizamos o Git com uma política de *Pull Requests*, onde o código é revisto antes de ser integrado na *branch* principal.

2.3 Integração Contínua (CI) e Quality Gates

Utilizámos o *GitHub Actions* para automatizar a verificação de qualidade a cada *push*. O sistema está estruturado em três eixos principais:

- **Backend:** Executa o *build* e os testes (unitários e de integração) de cada microserviço em paralelo. Implementámos *caching* de dependências Maven para otimizar o tempo de execução.
- **Frontend:** Garante a integridade da UI através de *linting* e testes *End-to-End* (E2E) com *Playwright*, validando o fluxo completo do utilizador.

Esta *pipeline* atua como um *Quality Gate*: o *merge* para a branch **main** é bloqueado automaticamente caso ocorra qualquer falha, garantindo a estabilidade contínua do software.

3 Objetivo

Este guia fornece as instruções necessárias para operar o Nexus Estates. Cobre a execução do sistema (via Docker ou ambiente de desenvolvimento), a *stack* tecnológica, os padrões de arquitetura adotados e a estratégia de validação através de testes.

Para mais detalhes operacionais, consulta:

- [README.md](#) (raiz)
- [backend/README.md](#)
- [frontend/README.md](#)

4 Como correr o projeto (Docker – Deploy em Produção)

O ecossistema foi desenhado para ser agnóstico ao ambiente. Através do GitHub Container Registry (GHCR), o *deploy* é efetuado de forma imutável e automática.

4.1 Comando de Deploy

Este comando descarrega o manifesto de orquestração e levanta todos os serviços (Gateway, Micro-serviços, Base de Dados e Frontend) sem necessidade de dependências locais.

Windows (PowerShell):

```
curl.exe -L -o docker-compose.deploy.yml "https://raw.githubusercontent.com/kanekiTakitos/Nexus-Estates/main/infrastructure/docker-compose.deploy.yml" ;  
docker compose -f docker-compose.deploy.yml up -d --pull always
```

Linux / Mac / WSL:

```
curl -L -o docker-compose.deploy.yml "https://raw.githubusercontent.com/kanekiTakitos/Nexus-Estates/main/infrastructure/docker-compose.deploy.yml" &&  
docker compose -f docker-compose.deploy.yml up -d --pull always
```

4.2 Acessos e Credenciais

Após o arranque, os serviços estarão operacionais nos seguintes *endpoints*:

- **Frontend:** <http://localhost:3000>
- **API Gateway:** <http://localhost:8080>
- **Swagger UI:** <http://localhost:8080/swagger-ui.html>

Credenciais de Teste:

Email: dev@nexus.com

Password: 12345

4.3 Gestão do Ciclo de Vida

Parar e limpar dados (Reset):

```
docker compose -f docker-compose.deploy.yml down -v
```

Remoção Completa (Cleanup Total): Remove contentores, volumes, imagens e o ficheiro YAML gerado pelo *curl*.

Linux / Mac / WSL:

```
docker compose -f docker-compose.deploy.yml down -v --rmi all && rm docker-compose  
.deploy.yml
```

Windows (PowerShell):

```
docker compose -f docker-compose.deploy.yml down -v --rmi all ; del docker-compose.deploy.yml
```

4.4 Arquitetura de Rede e Segurança

A infraestrutura utiliza uma rede isolada (**nexus-net**).

- **Isolamento:** Apenas o Frontend e o Gateway expõem portas ao *host*.
- **Service Discovery:** A comunicação *inter-service* é feita via DNS interno do Docker.
- **Segurança:** Bases de dados e *brokers* de mensagens estão inacessíveis fora da rede interna.

5 Ambiente de Desenvolvimento (DEV)

Para desenvolvimento ativo com *Hot Reload* e *debug*:

5.1 Infraestrutura Base

Levanta apenas os serviços de suporte (Base de Dados e RabbitMQ).

```
docker compose -f infrastructure/docker-compose.yml up -d
```

5.2 Backend e Frontend

Comandos para execução local dos serviços em modo de desenvolvimento.

```
# Terminal 1 (Java/Spring)
mvn -pl api-gateway -am spring-boot:run

# Terminal 2 (Next.js)
cd frontend && bun dev
```

Atualmente usamos o IDE de JetBrains que facilita a instalação e compilação.

6 Padrões de Projeto e Práticas (Backend)

6.1 Separação por Camadas (MVC + Service + Repository)

Os microserviços seguem uma organização modular e bem definida para garantir a separação de responsabilidades, facilitando a manutenção e a testabilidade individual. A estrutura de pastas segue o padrão:

- **controller**/: *Endpoints* REST e gestão da camada HTTP.
- **service**/: Implementação das regras de negócio e orquestração de fluxos.
- **repository**/: Camada de persistência e acesso a dados via Spring Data.
- **entity**/: Modelos JPA mapeados para as tabelas da base de dados.

- **dto/**: Contratos de comunicação (*Request/Response*) para evitar a exposição das entidades.
- **config/**: Configurações do ecossistema Spring (Security, RabbitMQ, Clientes HTTP).
- **messaging/**: Lógica de *consumers/publishers* para integração assíncrona.
- **exception/**: Gestão centralizada de erros e *Global Exception Handlers*.
- **audit/**: Auditoria de dados utilizando contexto de actor e Hibernate Envers.

6.2 Proxy Pattern (HTTP Declarativo entre Microserviços)

A comunicação *inter-service* utiliza clientes HTTP declarativos gerados em *runtime*, seguindo o **Proxy Pattern**. Esta abordagem reduz o acoplamento e centraliza os contratos de comunicação:

- As interfaces definidas em `client/NexusClients.java` descrevem os *endpoints* remotos.
- A classe `config/WebClientConfig.java` utiliza a `HttpServiceProxyFactory` para injetar os proxies dinâmicos em cada serviço, permitindo que a comunicação entre microserviços seja tão simples como chamar um método local.

6.3 Strategy Pattern (Ex.: Faturação no finance-service)

No *finance-service*, a lógica de emissão de faturas foi encapsulada através do **Strategy Pattern**. Isto permite que o fornecedor de faturação varie conforme o ambiente ou a configuração, sem alterar o `InvoiceOrchestrator`:

- **InvoiceProviderStrategy**: Interface que define o contrato para a emissão de documentos.
- **Implementações disponíveis**:
 - `MockInvoiceProviderStrategy` (Para testes e desenvolvimento local)
 - `MoloniInvoiceProviderStrategy` (Integração real via API REST)
 - `NoopInvoiceProviderStrategy` (Operação nula para desativação do serviço)

6.4 Resiliência (Circuit Breaker + Retry)

Para proteger o ecossistema contra falhas em cascata em chamadas externas, o *sync-service* utiliza a biblioteca *Resilience4j*:

- **Circuit Breaker**: Interrompe o tráfego para um serviço externo se este começar a falhar sistematicamente, permitindo a sua recuperação.
- **Retry with Backoff**: Efetua tentativas automáticas para erros transitórios, com tempos de espera incrementais.

6.5 Eventos Assíncronos e Consistência Eventual

A consistência de dados entre os diversos microserviços é gerida através de eventos, garantindo o desacoplamento temporal:

- **RabbitMQ**: Atua como o *message broker* para publicação e consumo de eventos (ex: *booking created*).

- **Saga Pattern (Coreografada):** Fluxos de trabalho distribuídos que asseguram que todos os serviços atingem um estado consistente após uma transação complexa.
- **Documentação:** A estrutura completa do projeto e os fluxos detalhados podem ser consultados no [README.md](#) da raiz do repositório.

7 Testes (backend)

Os testes do backend estão por microserviço em `<module>/src/test/java`. Para ver a lista completa (com nomes de testes), consulta `<module>/TESTS.md`.

7.1 api-gateway

- **Filtros e segurança:** validações do JWT, rotas públicas vs protegidas e regras de segurança do gateway.
- **Principais classes:** `AuthenticationFilterTest`, `RouteValidatorTest`, `JwtUtilTest`.

7.2 booking-service

- **API e validações:** criar reservas, validar payload, conflitos de datas e leituras por utilizador/propriedade.
- **Domínio:** regras (min/max noites, lead time), overlaps e consistência de estado.
- **Eventos/RabbitMQ:** publicar e consumir eventos, incluindo bloqueios de calendário.
- **Pagamentos:** integração com o `finance-service` via client declarativo.
- **Principais classes:** `BookingControllerTest`, `BookingServiceTest`, `BookingRepositoryTest`, `BookingEventPublisherTest`, `CalendarBlockConsumerTest`, `BookingPaymentServiceTest`.

7.3 property-service

- **CRUD e endpoints:** propriedades e amenities (criar/listar/editar/apagar), endpoints expandidos e histórico.
- **Regras e preços:** regras base, sazonalidade e cotações (quote).
- **Permissões:** ACL por níveis (`PRIMARY_OWNER/MANAGER/STAFF`) e validações.
- **Upload:** parâmetros de upload e integração com Clouinary.
- **Auditoria:** `ActorContext` e Envers.

7.4 sync-service

- **Config e filas:** bindings RabbitMQ + DLQ e testes de configuração.
- **Integração externa:** cliente HTTP, autenticação externa e resiliência (inclui testes de integração).
- **Ferramentas admin:** endpoints para DLQ e parsing/apply de ICS.
- **Webhooks e chat:** validações de assinatura, dispatch de webhooks e persistência/notificações.

7.5 finance-service

- **Pagamentos:** criação/confirmação e persistência de pagamentos.
- **Webhooks Stripe:** validação de assinatura e idempotência.
- **Faturação:** orquestração de invoice e estratégias (mock/moloni/noop).

7.6 user-service

- **Auth:** registo/login e JWT.
- **Segurança:** filtro JWT, roles e headers do gateway.
- **Recuperação de password:** controllers e serviço (tokens, expiração, privacidade).
- **Integrações:** endpoints para integrações externas.
- **Auditoria:** `ActorContext` e Envers.

7.7 Como correr os testes (backend)

Para validar a integridade e o comportamento dos microserviços, debes executar os testes a partir da pasta `backend`.

Execução global (todos os módulos):

```
mvn test
```

COPY

Execução por módulo específico: Utiliza as flags `-pl` (*project list*) e `-am` (*also make*) para testar apenas um serviço e as suas dependências internas:

```
mvn -pl booking-service -am test
```

COPY

8 Testes (frontend) — integração/E2E (Playwright)

Os testes do frontend estão em `frontend/e2e/api`. Eles testam os fluxos principais via API Gateway (por defeito `http://localhost:8080/api`).

- `auth.spec.ts`: registo/login, permissões por role, perfil, integrações e webhooks (inclui casos sem token e casos de segurança como IDOR).
- `properties.spec.ts`: CRUD de propriedades, validações, regras/sazonalidade, permissões (ACL) e endpoints de upload.
- `bookings.spec.ts`: criar reservas, validar conflitos de datas/overlaps, bloqueios e listagens do utilizador.
- `amenities.spec.ts`: CRUD de amenities.

8.1 Como correr testes do frontend

Pré-condição: O *backend* deve estar acessível e funcional (ex.: através do *deploy* Docker com o *gateway* a responder em `http://localhost:8080`).

Na pasta `frontend`, utiliza o seguinte comando para executar a suite de testes *End-to-End* (E2E) em modo *headless*:

```
bun test:e2e
```

COPY

Para depuração ou visualização da execução em tempo real, podes utilizar o **Modo UI** do Playwright:

```
bun test:e2e:ui
```

COPY

Referências

- [1] Google Cloud. *O que é a arquitetura de microsserviços?*.
Disponível em: <https://cloud.google.com/learn/what-is-microservices-architecture?hl=pt-BR>
- [2] Docker. *Docker Documentation*.
Disponível em: <https://docs.docker.com/>
- [3] Vercel. *Next.js - The React Framework for the Web*.
Disponível em: <https://nextjs.org/>
- [4] Bun. *Fast all-in-one JavaScript runtime*.
Disponível em: <https://bun.com/>
- [5] Tailwind Labs. *Tailwind CSS*.
Disponível em: <https://tailwindcss.com/>
- [6] Microsoft. *TypeScript*.
Disponível em: <https://www.typescriptlang.org/>
- [7] VMware. *Spring Boot*.
Disponível em: <https://spring.io/projects/spring-boot>
- [8] TPointTech. *Java Tutorial*.
Disponível em: <https://www.tpointtech.com/java-tutorial>
- [9] PostgreSQL Global Development Group. *PostgreSQL Documentation*.
Disponível em: <https://www.postgresql.org/docs/>
- [10] YouTube (Amigoscode). *Spring Boot Tutorial*.
Disponível em: <https://www.youtube.com/watch?v=gJrjgg1KVL4>
- [11] YouTube (Bouali Ali). *Spring Boot Tutorial - Full Course*.
Disponível em: https://www.youtube.com/watch?v=YY_hf0FOIcU